Lab 2 report

October 24, 2026

Contents

1 Phase1 **2**
- 1.1 关键汇编 . . . 2
- 1.2 逐行代码推理过程 . . . 4
- 1.3 额外 seed 验证记录 . . . 4

2 Phase2 **5**
- 2.1 关键汇编 . . . 5
- 2.2 逐行代码推理过程 . . . 7
- 2.3 额外 seed 验证记录 . . . 7

3 Phase3 **8**
- 3.1 关键汇编 . . . 8
- 3.2 逐行代码推理过程 . . . 10
- 3.3 额外 seed 验证记录 . . . 10

4 Phase4 **11**
- 4.1 关键汇编 . . . 11
- 4.2 逐行代码推理过程 . . . 13
- 4.3 额外 seed 验证记录 . . . 13

5 Phase5 **14**
- 5.1 关键汇编 . . . 14
- 5.2 逐行代码推理过程 . . . 16
- 5.3 额外 seed 验证记录 . . . 16

6 Phase6 **17**
- 6.1 关键汇编 . . . 17
- 6.2 逐行代码推理过程 . . . 19
- 6.3 额外 seed 验证记录 . . . 19

7 Honor **20**
- 7.1 概要分析 . . . 20

8 思考与总结 **21**

1 Phase1

1.1 关键汇编

1. 关键汇编代码样例 phase1 可以拆出一个计算汇编，然后 call 调用了三个函数，所以综合一起有四个关键的汇编。

```
0000000000401fe0 <p1_compute>:
  401fe0: endbr64
  401fe4: push   %rbp
  401fe5: mov    %rdi,%rax
  401fe8: mov    %rdi,%rdx
  401feb: mov    %edi,%r8d
  401fee: shr    $0x20,%rax
  401ff2: shr    $0x10,%rdx
  401ff6: mov    %eax,%r9d
  401ff9: mov    %eax,%esi
  401ffb: mov    %rsp,%rbp
  401ffe: call   401fb0 <p1_step0>
  402003: mov    %rdx,%rsi
  402006: mov    %r9d,%edi
  402009: mov    %eax,%ecx
  40200b: call   401fc0 <p1_step1>
  402010: mov    %r8d,%esi
  402013: mov    %eax,%edi
  402015: call   401fd0 <p1_step2>
  40201a: pop    %rbp
  40201b: xor    %ecx,%eax
  40201d: ret
```

Figure 1: phase1 计算汇编

```
0000000000401fb0 <p1_step0>:
  401fb0: endbr64
  401fb4: xor    $0x72bd,%esi
  401fba: lea    (%rsi,%rdi,4),%eax
  401fbd: ret
```

Figure 2: phase1 辅助函数 step0

```
0000000000401fc0 <p1_step1>:
  401fc0: endbr64
  401fc4: lea    (%rdi,%rdi,4),%eax
  401fc7: add    $0x4246,%esi
  401fcd: xor    %esi,%eax
  401fcf: ret
```

Figure 3: phase1 辅助函数 step1

```
0000000000401fd0 <p1_step2>:
  401fd0: endbr64
  401fd4: lea    (%rdi,%rdi,4),%eax
  401fd7: add    %esi,%eax
  401fd9: xor    $0xce9d,%eax
  401fde: ret
```

Figure 4: phase1 辅助函数 step2

2. 恢复出的计算关系 令 C 表示 64 位的 `challenge`。根据 `mov` 和 `shr` 指令，可将其拆分为

$$L = (uint32_t)C, \qquad M = (uint32_t)(C \gg 16), \qquad H = (uint32_t)(C \gg 32). \quad (1)$$

其中，`lea (%rsi,%rdi,4),%eax` 等价于 `eax` $=$ `esi` $+ 4$`edi`。由三个辅助函数的汇编可依次恢复出

$$S_0 = (H \oplus \texttt{0x72bd}) + 4L, \quad (2)$$
$$S_1 = 5H \oplus (M + \texttt{0x4246}), \quad (3)$$
$$S_2 = (5S_1 + L) \oplus \texttt{0xce9d}. \quad (4)$$

最后的 `xor %ecx,%eax` 将 S_2 与先前保存在 `%ecx` 中的 S_0 异或，因此正确输入为

$$\boxed{a_0 = S_2 \oplus S_0}. \quad (5)$$

3. 控制流图呈现

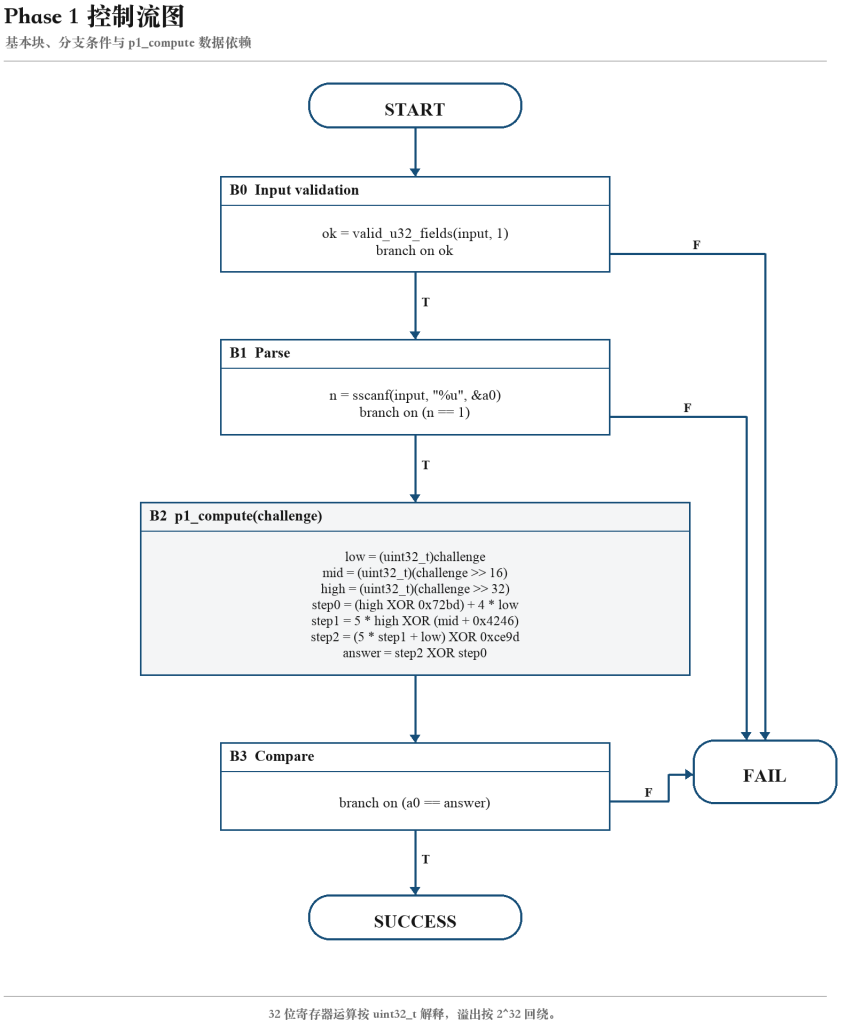


Figure 5: phase1 控制流图

4. 运行成功截图

```
第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0x28efe333b266f103
Input (unsigned decimal integers):
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
```

Figure 6: phase1 运行成功截图

1.2 逐行代码推理过程

左侧为编译语言翻译，右侧为具体达成结果，以下是具体的推理思路。

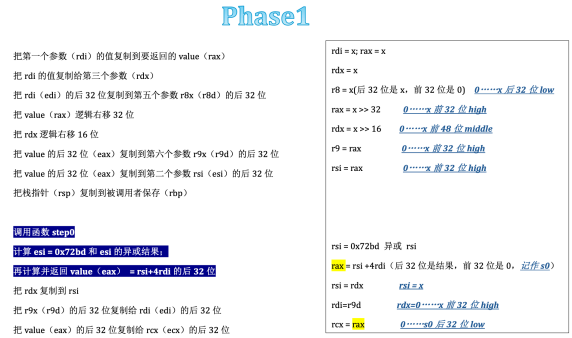


Figure 7: 推理部分

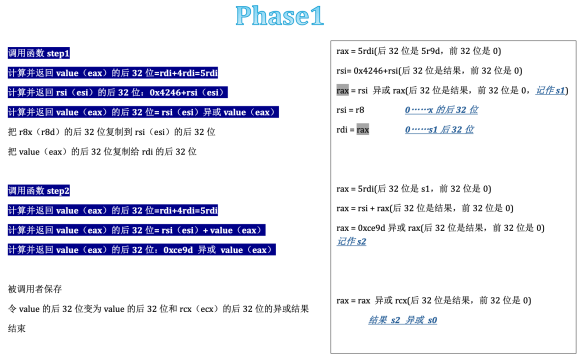


Figure 8: 推理部分 (深蓝色为辅助函数及其展开表达)

1.3 额外 seed 验证记录

采用了公开 10 个 seed 以外的 314159 进行测试，测试成功。



Figure 9: phase1 的额外 seed 验证

2 Phase2

2.1 关键汇编

1. 关键汇编代码样例 phase2 前面一大部分都在检查数组元素的输入，主要的是后半部分分别验证 a0 和 a1-a5 的验证汇编。

```
0000000000402080 <phase_02>:
  4020ea: mov    %r13,%rax
  4020ed: mov    -0x40(%rbp),%esi
  4020f0: shr    $0x6,%rax
  4020f4: and    $0x7f,%eax
  4020f7: add    $0x5,%eax
  4020fa: cmp    %eax,%esi
  4020fc: jne    4020a5 <phase_02+0x25>
  4020fe: mov    $0x1,%edx
  402103: lea    0x2(%rdx),%ecx
  402106: mov    %r13,%rax
  402109: shr    %cl,%rax
  40210c: lea    (%rsi,%rsi,4),%ecx
  40210f: mov    -0x40(%rbp,%rdx,4),%esi
  402113: and    $0x3f,%eax
  402116: add    %edx,%eax
  402118: xor    %ecx,%eax
  40211a: cmp    %eax,%esi
  40211c: jne    4020a5 <phase_02+0x25>
  40211e: add    $0x1,%rdx
  402122: cmp    $0x6,%rdx
  402126: jne    402103 <phase_02+0x83>
  402128: jmp    4020a7 <phase_02+0x27>
```

Figure 10: phase2 验证汇编

2. 恢复出的计算关系 Phase 2 要求输入六个无符号整数 $a_0, \ldots, a_5$。首项由以下指令计算：

$$\texttt{shr \$0x6,\%rax}, \quad \texttt{and \$0x7f,\%eax}, \quad \texttt{add \$0x5,\%eax}.$$

因此

$$\boxed{a_0 = ((C \gg 6)\ \&\ \texttt{0x7f}) + 5}. \tag{6}$$

循环从 $i = 1$ 开始。进入每轮循环时，$\texttt{rdx} = i$，$\texttt{esi} = a_{i-1}$。指令

```
lea (%rsi,%rsi,4),%ecx
```

计算 $5a_{i-1}$，而

```
mov -0x40(%rbp,%rdx,4),%esi
```

从数组地址 $\texttt{rbp} - \texttt{0x40} + 4i$ 读取当前输入 a_i。

结合移位、掩码、加法和异或指令，可恢复出递推关系

$$\boxed{a_i = (((C \gg (i+2))\ \&\ \texttt{0x3f}) + i) \oplus 5a_{i-1}, \qquad 1 \le i \le 5.} \tag{7}$$

3. 控制流图呈现

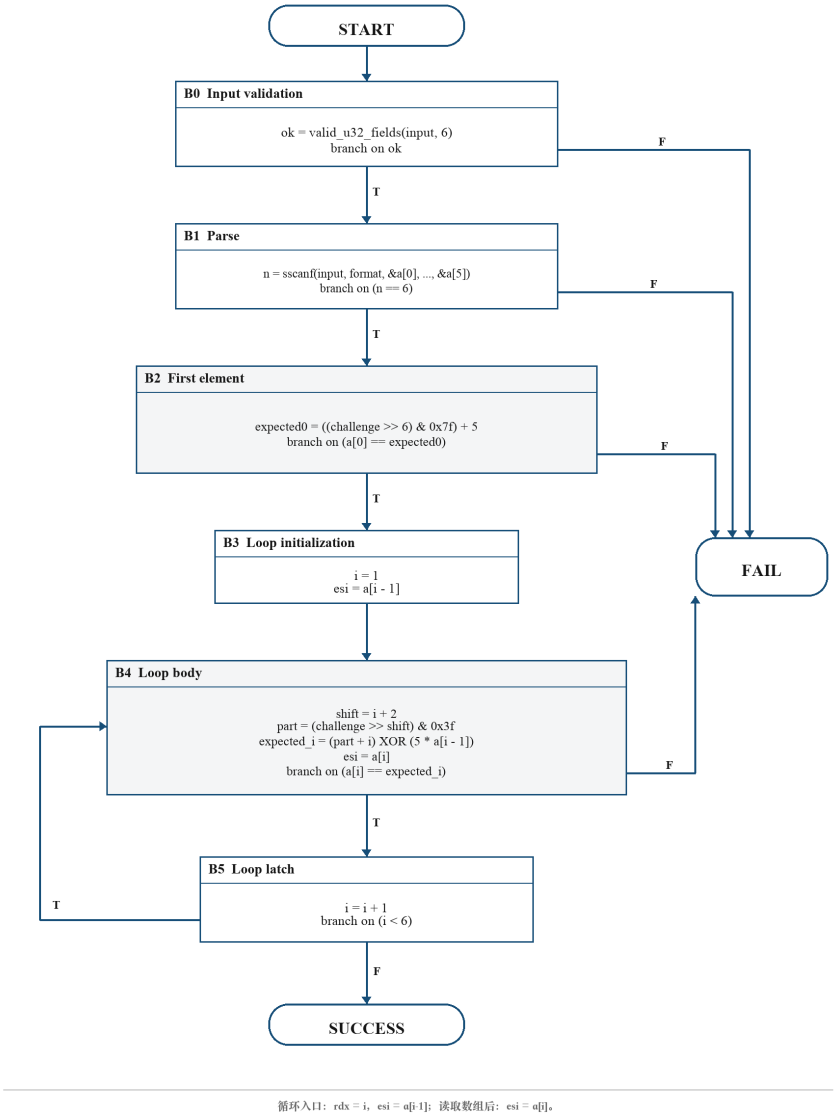


Figure 11: phase2 控制流图

4. 运行成功截图

```
第 2 关 · 钟楼
钟楼没有钟声，齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角，司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
```

Figure 12: phase2 运行成功截图

2.2 逐行代码推理过程

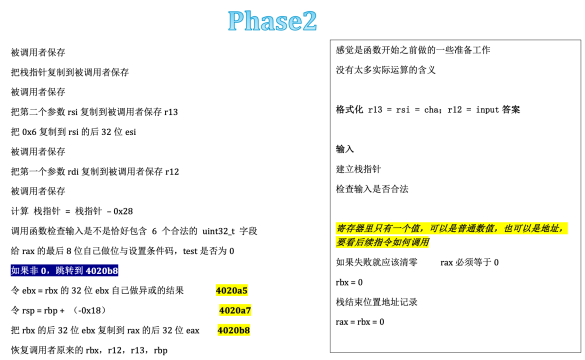


Figure 13: 推理部分 (深蓝色为对应循环判断部分，黄色加粗数字是对应跳转的位置)

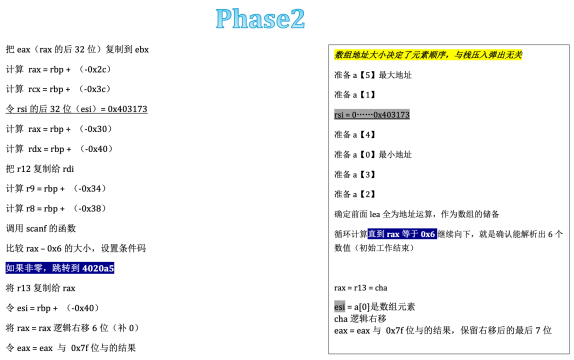


Figure 14: 推理部分

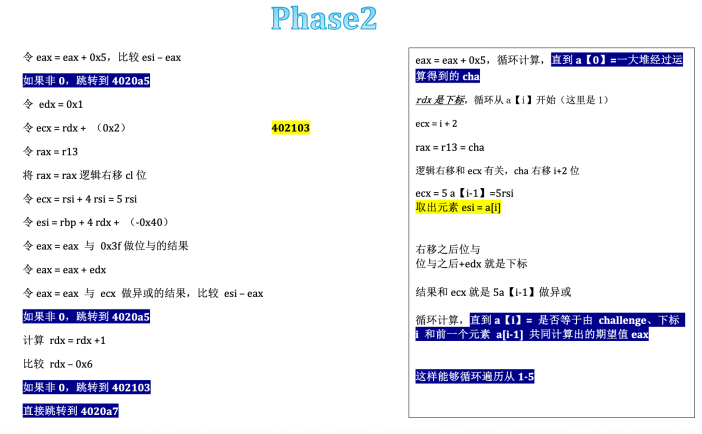


Figure 15: 推理部分

2.3 额外 seed 验证记录

采用了公开 10 个 seed 以外的 271828 进行测试，测试成功。



Figure 16: phase2 的额外 seed 验证

3 Phase3

3.1 关键汇编

1. 关键汇编代码样例 phase3 的前面也是在做读入，和两步具体的计算结果，然后进入 switch 函数对应到不同 case 中进行计算，最后在进行验证。

```
40228c: 44 89 e7              mov    %r12d,%edi
40228f: 31 c0                 xor    %eax,%eax
402291: 83 e7 07              and    $0x7,%edi
402294: 39 7d e8              cmp    %edi,-0x18(%rbp)
402297: 75 cc                 jne    402265 <phase_03+0x25>
402299: 4c 89 e0              mov    %r12,%rax
40229c: 4c 89 e6              mov    %r12,%rsi
40229f: 48 c1 e8 03           shr    $0x3,%rax
4022a3: 48 c1 ee 20           shr    $0x20,%rsi
4022a7: 31 c6                 xor    %eax,%esi
4022a9: e8 32 ff ff ff        call   4021e0 <p3_dispatch>
4022ae: 39 45 ec              cmp    %eax,-0x14(%rbp)
4022b1: 0f 94 c0              sete   %al
4022b4: eb af                 jmp    402265 <phase_03+0x25>
```

Figure 17: 主体汇编

```
0000000000402e0 <p3_dispatch>:
  4021e0: f3 0f 1e fa           endbr64
  4021e4: 89 f8                 mov    %edi,%eax
  4021e6: 89 f7                 mov    %esi,%edi
  4021e8: 83 f8 07              cmp    $0x7,%eax
  4021eb: 77 48                 ja     402235 <p3_dispatch+0x55>
  4021ed: 3e ff 24 c5 e0 57 40  notrack jmp *0x4057e0(,%rax,8)
  4021f4: 00
  4021f5: 0f 1f 00              nopl   (%rax)
  4021f8: eb d6                 jmp    4021d0 <p3_case_7>
  4021fa: 66 0f 1f 44 00 00     nopw   0x0(%rax,%rax,1)
  402200: eb ae                 jmp    4021b0 <p3_case_6>
  402202: 66 0f 1f 44 00 00     nopw   0x0(%rax,%rax,1)
  402208: eb 96                 jmp    4021a0 <p3_case_5>
  40220a: 66 0f 1f 44 00 00     nopw   0x0(%rax,%rax,1)
  402210: e9 5b ff ff ff        jmp    402170 <p3_case_3>
  402215: 0f 1f 00              nopl   (%rax)
  402218: e9 23 ff ff ff        jmp    402140 <p3_case_1>
  40221d: 0f 1f 00              nopl   (%rax)
  402220: e9 0b ff ff ff        jmp    402130 <p3_case_0>
  402225: 0f 1f 00              nopl   (%rax)
  402228: e9 33 ff ff ff        jmp    402160 <p3_case_2>
  40222d: 0f 1f 00              nopl   (%rax)
  402230: e9 5b ff ff ff        jmp    402190 <p3_case_4>
  402235: 31 c0                 xor    %eax,%eax
  402237: c3                    ret
```

Figure 18: switch 函数汇编（具体计算展开在思路中）

2. 恢复出的计算关系 设 C 为传入的 64 位无符号整数 `challenge`，并用 $[x]_{32}$ 表示保留 x 的低 32 位。程序首先计算

$$s_0 = [C]_{32} \,\&\, \texttt{0x7},$$

因此 $s_0 \in \{0, 1, \ldots, 7\}$，既是第一个输入值，也是跳转表的 case 编号。第一个输入必须满足

$$a_0 = s_0.$$

随后计算各分支共用的参数

$$s_1 = [C \gg 32]_{32} \oplus [C \gg 3]_{32},$$

其中 $\oplus$ 表示按位异或。跳转表根据 s_0 选择分支，并以 s_1 作为计算参数。恢复出的分段函数为

$$F(s_0, s_1) = \begin{cases} 5s_1 + \texttt{0x438fa}, & s_0 = 0, \\ (4s_1) \oplus \texttt{0xd4ef}, & s_0 = 1, \\ 5s_1 + \texttt{0x5d15}, & s_0 = 2, \\ (s_1 \oplus \texttt{0x67e5}) + (s_1 \gg 2), & s_0 = 3, \\ 5s_1 + \texttt{0x9cb0}, & s_0 = 4, \\ (2s_1) \oplus \texttt{0xe4d5}, & s_0 = 5, \\ (s_1 \oplus \texttt{0xdade}) + (s_1 \gg 2), & s_0 = 6, \\ 5s_1 + \texttt{0x315f1}, & s_0 = 7. \end{cases}$$

所有计算均采用 32 位无符号整数运算，即结果按模 2^{32} 回绕。因此 Phase 3 最终应输出

$$\boxed{a_0 = s_0, \qquad a_1 = F(s_0, s_1)}.$$

3. 控制流图呈现

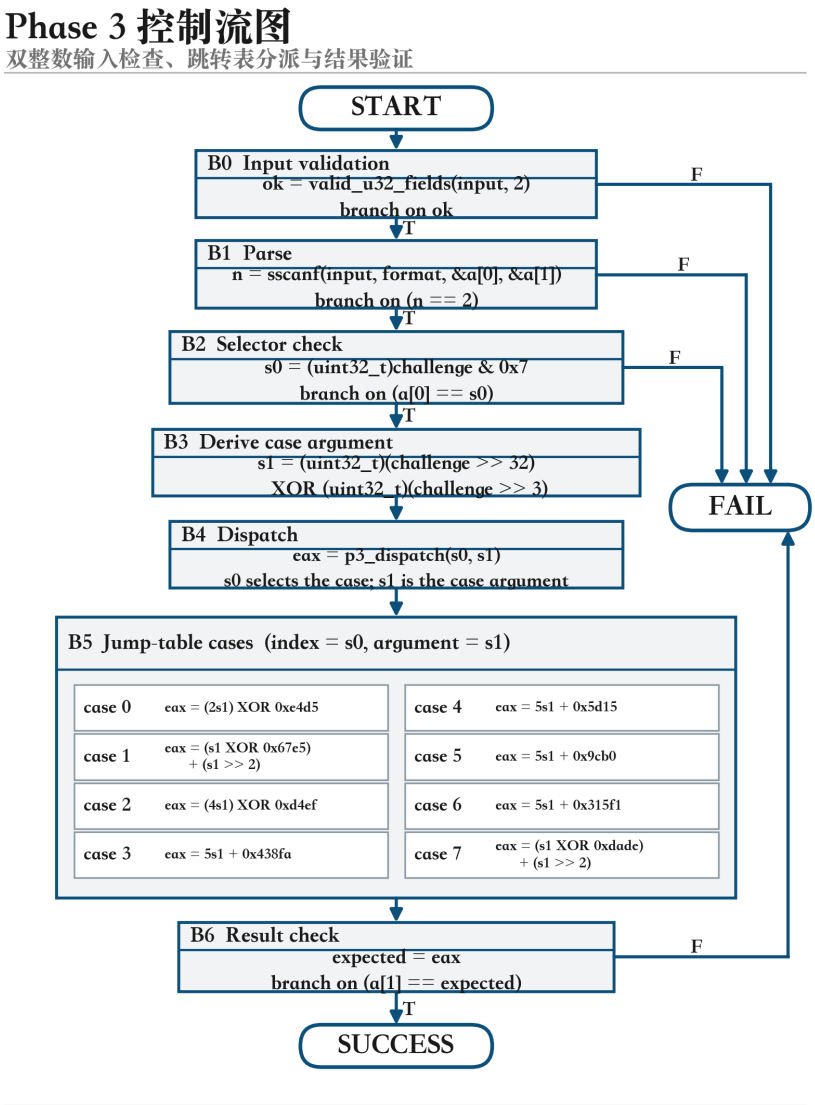


Figure 19: phase3 控制流图

4. 运行成功截图

```
第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0xc8f302267d8e98d7
Input (unsigned decimal integers):
Phase 3 defused.

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]

Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

Figure 20: phase3 运行成功截图

3.2 逐行代码推理过程



Figure 21: 推理部分

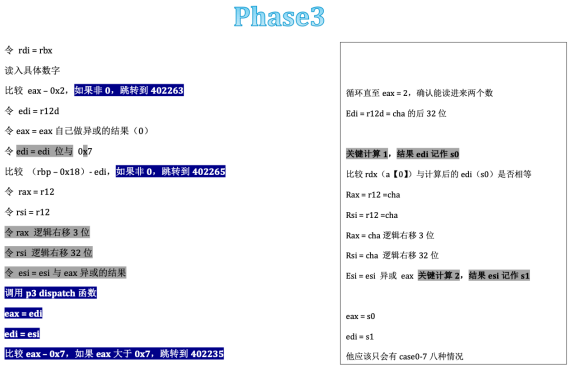


Figure 22: 推理部分

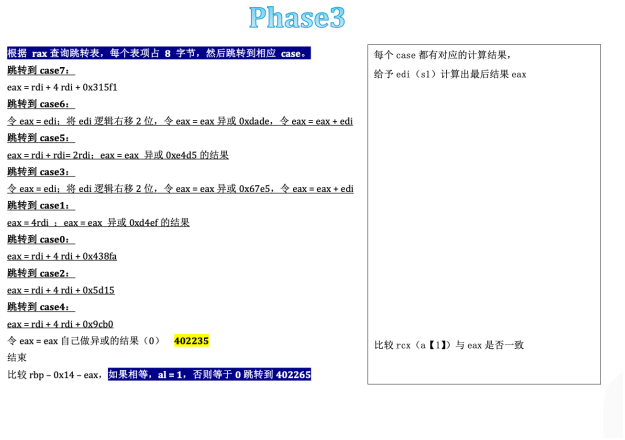


Figure 23: 推理部分

3.3 额外 seed 验证记录

采用了公开 10 个 seed 以外的 314159265 进行测试，测试成功。



Figure 24: phase3 额外 seed 验证

4 Phase4

4.1 关键汇编

1. 关键汇编代码样例

```
0000000000402330 <phase_04>:
  402378: 4c 89 e1              mov    %r12,%rcx
  40237b: 4c 89 e0              mov    %r12,%rax
  40237e: 44 89 e7              mov    %r12d,%edi
  402381: 31 f6                 xor    %esi,%esi
  402383: 48 c1 e8 20           shr    $0x20,%rax
  402387: 48 c1 e9 07           shr    $0x7,%rcx
  40238b: 83 e7 7f              and    $0x7f,%edi
  40238e: ba 7f 00 00 00        mov    $0x7f,%edx
  402393: 31 c1                 xor    %eax,%ecx
  402395: e8 26 ff ff ff        call   4022c0 <p4_walk>
  40239a: 39 45 ec              cmp    %eax,-0x14(%rbp)
  40239d: 0f 94 c0              sete   %al
  4023a0: 48 83 c4 10           add    $0x10,%rsp
  4023a4: 5b                    pop    %rbx
  4023a5: 41 5c                 pop    %r12
  4023a7: 5d                    pop    %rbp
  4023a8: c3                    ret
```

Figure 25: 计算汇编（进入递归函数之前的运算）

```
00000000004022c0 <p4_walk>:
  4022c4: 89 f8                 mov    %edi,%eax
  4022c6: 89 d7                 mov    %edx,%edi
  4022c8: 29 f7                 sub    %esi,%edi
  4022ca: d1 ef                 shr    $1,%edi
  4022cc: 01 f7                 add    %esi,%edi
  4022ce: 39 c7                 cmp    %eax,%edi
  4022d0: 74 4e                 je     402320 <p4_walk+0x60>
  4022d2: 55                    push   %rbp
  4022d3: 48 89 e5              mov    %rsp,%rbp
  4022d6: 53                    push   %rbx
  4022d7: 8d 1c 0f              lea    (%rdi,%rcx,1),%ebx
  4022da: 48 83 ec 08           sub    $0x8,%rsp
  4022de: 39 f8                 cmp    %edi,%eax
  4022e0: 72 1e                 jb     402300 <p4_walk+0x40>
  4022e2: 8d 77 01              lea    0x1(%rdi),%esi
  4022e5: 89 c7                 mov    %eax,%edi
  4022e7: e8 d4 ff ff ff        call   4022c0 <p4_walk>
  4022ec: 8d 04 80              lea    (%rax,%rax,4),%eax
  4022ef: 31 d8                 xor    %ebx,%eax
  4022f1: 48 8b 5d f8           mov    -0x8(%rbp),%rbx
  4022f5: c9                    leave
  4022f6: 83 f0 13              xor    $0x13,%eax
  4022f9: c3                    ret
  4022fa: 66 0f 1f 44 00 00     nopw   0x0(%rax,%rax,1)
  402300: 8d 57 ff              lea    -0x1(%rdi),%edx
  402303: 89 c7                 mov    %eax,%edi
  402305: e8 b6 ff ff ff        call   4022c0 <p4_walk>
  40230a: 8d 04 80              lea    (%rax,%rax,4),%eax
  40230d: 31 d8                 xor    %ebx,%eax
  40230f: 48 8b 5d f8           mov    -0x8(%rbp),%rbx
  402313: c9                    leave
  402314: 83 f0 1e              xor    $0x1e,%eax
  402317: c3                    ret
```

Figure 26: p4 递归函数主要汇编

2. 恢复出的计算关系 记传入的 challenge 为 C。由 Phase 4 主函数可得，递归函数的四个初始参数分别为

$$t = C \,\&\, 0x7f, \qquad l = 0, \qquad h = 127,$$

以及

$$k = \left((C \gg 7) \bmod 2^{32}\right) \oplus \left((C \gg 32) \bmod 2^{32}\right).$$

在递归函数中，每一层首先计算区间中点

$$m = l + \left\lfloor \frac{h-l}{2} \right\rfloor.$$

令 $F(t,l,h,k)$ 表示 `p4_walk` 的返回值，则其递归关系为

$$F(t,l,h,k) = \begin{cases} (m+2) \oplus k, & t = m, \\ (5F(t,l,m-1,k) \oplus (m+k)) \oplus 0x1e, & t < m, \\ (5F(t,m+1,h,k) \oplus (m+k)) \oplus 0x13, & t > m. \end{cases}$$

因此，Phase 4 所要求的唯一输入整数为

$$\boxed{a_0 = F(t,0,127,k)}$$

3. 控制流图呈现

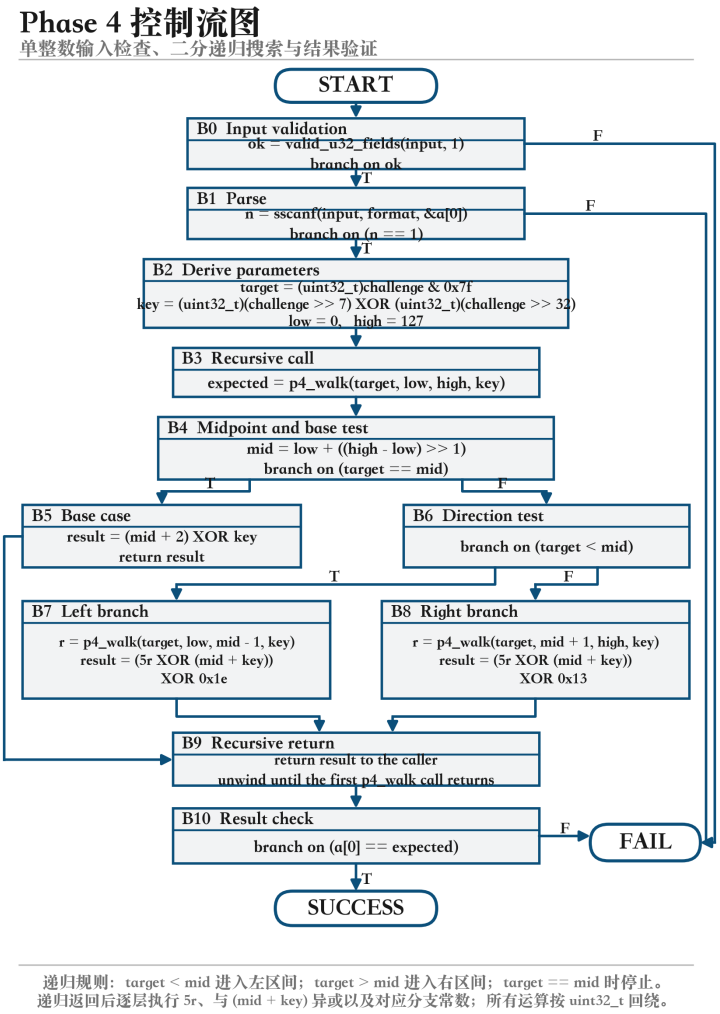


Figure 27: phase4 控制流图

4. 运行成功截图

```
第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x09bc585a244823f2
Input (unsigned decimal integers):
Phase 4 defused.

[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]

Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
```

Figure 28: phase4 运行成功截图

4.2 逐行代码推理过程

phase4 很难从逐行代码分析出主要函数框架，所以要在逐行分析前插入一下递归汇编汇编内部最重要的几个点：

1. 主要的四个参数：target，用于和每个区间中点去比较的参数；high 区间上界；low 区间下界；mid 区间中点；

2. 从上之下的递推过程：每一轮要确定区间，区间中点，用 target 和 mid 做比较，看看是进入 mid 左边的区间还是右边的区间，然后跳回到函数开头更新新的区间和区间中点，直到 mid 和 target 相等；

3. 从下至上的递推过程：在 mid 和 target 一致的时候，要先去函数底部做一个运算得到结果 rax，然后再从下至上回到刚才递归的层读取保存好的信息，用 rax 参与运算；

4. 每一层的参数保存：汇编在递归调用前执行`lea (%rdi,%rcx,1),%ebx`，保存本层的 mid 和 key，随后执行`push %rbx`，把上一层的 `rbx` 保存到栈里。下一层计算结束时，会恢复它`mov -0x8(%rbp),%rbx`，因此在从下至上递推的时候都能读取每一层的对应参数。

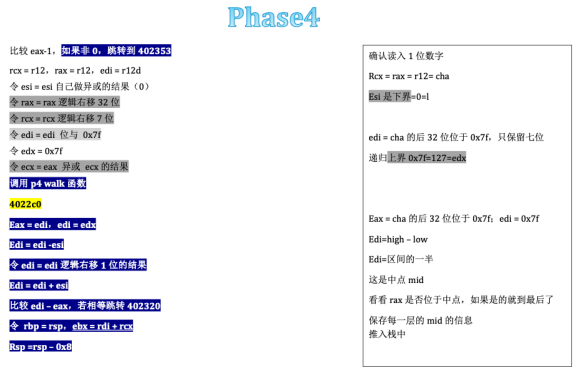


Figure 29: 核心推理过程

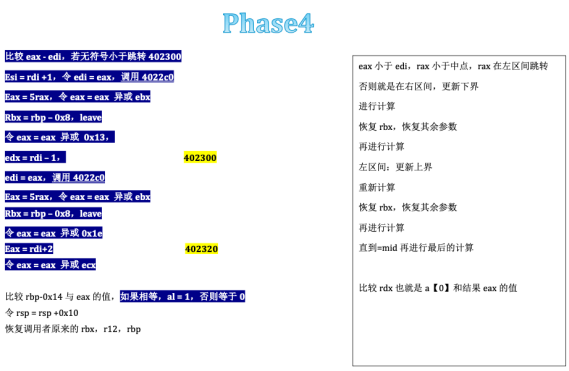


Figure 30: 核心推理过程

4.3 额外 seed 验证记录

采用了公开 10 个 seed 以外的 314159 进行测试，测试成功。



Figure 31: phase4 额外 seed 验证

5 Phase5

5.1 关键汇编

1. 关键汇编代码样例 除去 traverse 函数以外，指针表内部每个节点的字段值（field8）并不是显而易见写在明面的，需要我们去运行 gdb 进行动态查找，确认 0-7 对应的每个节点的地址以及他们定义好的对应的字段值。

```
00000000004023b0 <p5_traverse>:
  4023b0: f3 0f 1e fa           endbr64
  4023b4: 48 89 f8              mov    %rdi,%rax
  4023b7: 48 89 f9              mov    %rdi,%rcx
  4023ba: 49 89 f0              mov    %rsi,%r8
  4023bd: 83 e0 07              and    $0x7,%eax
  4023c0: 48 c1 e9 03           shr    $0x3,%rcx
  4023c4: 48 8b 14 c5 20 5b 40  mov    0x405b20(,%rax,8),%rdx
  4023cb: 00
  4023cc: 48 89 f8              mov    %rdi,%rax
  4023cf: 48 c1 ef 20           shr    $0x20,%rdi
  4023d3: 48 c1 e8 06           shr    $0x6,%rax
  4023d7: 31 f8                 xor    %edi,%eax
  4023d9: 8b 72 08              mov    0x8(%rdx),%esi
  4023dc: 89 cf                 mov    %ecx,%edi
  4023de: 35 61 95 00 00        xor    $0x9561,%eax
  4023e3: 03 42 0c              add    0xc(%rdx),%eax
  4023e6: 8d 04 80              lea    (%rax,%rax,4),%eax
  4023e9: 41 89 30              mov    %esi,(%r8)
  4023ec: 31 f0                 xor    %esi,%eax
  4023ee: 83 e7 03              and    $0x3,%edi
  4023f1: 74 1d                 je     402410 <p5_traverse+0x60>
  4023f3: b9 01 00 00 00        mov    $0x1,%ecx
  4023f8: 48 8b 12              mov    (%rdx),%rdx
  4023fb: 83 c1 01              add    $0x1,%ecx
  4023fe: 8b 72 08              mov    0x8(%rdx),%esi
  402401: 03 42 0c              add    0xc(%rdx),%eax
  402404: 8d 04 80              lea    (%rax,%rax,4),%eax
  402407: 41 89 30              mov    %esi,(%r8)
  40240a: 31 f0                 xor    %esi,%eax
  40240c: 39 cf                 cmp    %ecx,%edi
  40240e: 73 e8                 jae    4023f8 <p5_traverse+0x48>
  402410: c3                    ret
```

Figure 32: phase5 traverse 函数

```
(gdb) x/8gx 0x405b20
0x405b20 <p5_heads>:    0x0000000000405b70      0x0000000000405c00
0x405b30 <p5_heads+16>: 0x0000000000405bf0      0x0000000000405bd0
0x405b40 <p5_heads+32>: 0x0000000000405be0      0x0000000000405c10
0x405b50 <p5_heads+48>: 0x0000000000405b60      0x0000000000405b80
(gdb) set $node = *(void **)0x405b20
(gdb) p/x $node
$1 = 0x405b70
(gdb) x/1gx $node
0x405b70 <p5_nodes+16>: 0x0000000000405c00
(gdb) x/2uw ((char *)$node + 8)
0x405b78 <p5_nodes+24>: 793     3534
```

Figure 33: phase5 gdb 运行结果示例（从 405b20 开始的指针表）

2. 恢复出的计算关系 设 challenge 为 C，按链的实际遍历顺序，将节点偏移 8 和 12 处的字段分别记为 v_j、w_j。根据汇编，起点和移动次数为

$$s = C \,\&\, 7, \qquad t = (C \gg 3) \,\&\, 3.$$

起始节点也参与计算，因此共处理 $t+1$ 个节点。

记 $L_{32}(x) = x \bmod 2^{32}$，$\oplus$ 表示按位异或。初始值为

$$A_0 = L_{32}(C \gg 6) \oplus L_{32}(C \gg 32) \oplus \texttt{0x9561}.$$

每访问一个节点，执行

$$A_{k+1} = L_{32}(5(A_k + w_{s+k})) \oplus v_{s+k}, \quad k = 0, \ldots, t.$$

函数同时将当前节点的 v 字段写入输出地址，最终留下最后访问节点的字段值。因此，两个口令整数为

$$\boxed{a_0 = A_{t+1}, \qquad a_1 = v_{s+t}.}$$

3. 控制流图呈现

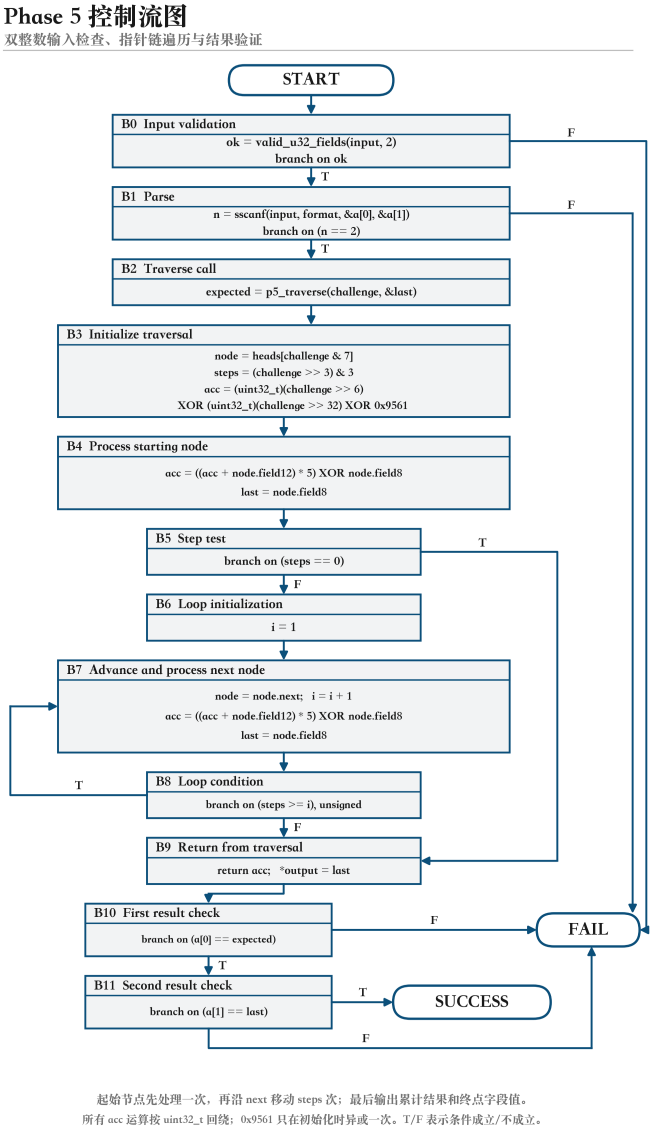


Figure 34: phase5 控制流图

4. 运行成功截图

```
第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边，
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xde4431fa3c80db06
Input (unsigned decimal integers):
Phase 5 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 已解除] [6 待解除]

Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
```

Figure 35: phase5 运行成功截图

5.2 逐行代码推理过程

需要注意的关键点：提前设置出一个地址去存放后续函数不断取出的地址位置，设计大的指针表，以及遍历步数限制，eax 累计结果和a[0]最后做比较，a[2] 不断覆盖，留下最后节点的字段值和a[1]做比较

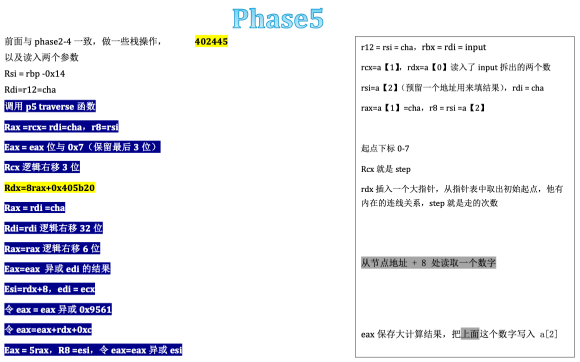


Figure 36: 推理过程

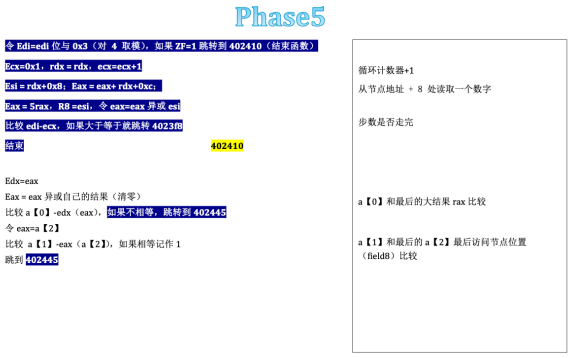


Figure 37: 推理过程

Table 1: Phase 5：起始指针表、节点数据

下标	节点地址	对应节点字段	下一节点
0	0x405b70	$v = 793$, $w = 3534$	next = 0x405c00
1	0x405c00	$v = 1872$, $w = 64114$	next = 0x405bf0
2	0x405bf0	$v = 2645$, $w = 11036$	next = 0x405bd0
3	0x405bd0	$v = 1119$, $w = 50920$	next = 0x405be0
4	0x405be0	$v = 2110$, $w = 16434$	next = 0x405c10
5	0x405c10	$v = 319$, $w = 27533$	next = 0x405b60
6	0x405b60	$v = 70$, $w = 34157$	next = 0x405b80
7	0x405b80	$v = 1292$, $w = 55659$	next = 0x405ba0
以上为 p5_heads 的八个表项；以下为后续节点，编号按链顺序排列。			
8	0x405ba0	$v = 597$, $w = 52460$	next = 0x405bb0
9	0x405bb0	$v = 2870$, $w = 33036$	next = 0x405bc0
10	0x405bc0	$v = 1586$, $w = 9192$	next = 0x405b90
11	0x405b90	$v = 2316$, $w = 35085$	next = NULL

5.3 额外 seed 验证记录

采用了公开 10 个 seed 以外的 13579 进行测试，测试成功。



Figure 38: phase5 的额外 seed 验证

6 Phase6

6.1 关键汇编

1. 关键汇编代码样例 phase6 内部也有一个指针表，在 0x405dc0 的位置，需要运行 gdb 来察看具体内部每个 case 对应的函数是什么样子的。

```
0000000000402510 <p6_dispatch>:
  402510: f3 0f 1e fa           endbr64
  402514: 55                    push   %rbp
  402515: 48 89 e5              mov    %rsp,%rbp
  402518: 53                    push   %rbx
  402519: 48 89 fb              mov    %rdi,%rbx
  40251c: 48 c1 ef 06           shr    $0x6,%rdi
  402520: 48 89 d8              mov    %rbx,%rax
  402523: 48 c1 e8 20           shr    $0x20,%rax
  402527: 31 c7                 xor    %eax,%edi
  402529: 48 89 d8              mov    %rbx,%rax
  40252c: 48 83 ec 08           sub    $0x8,%rsp
  402530: 83 e3 38              and    $0x38,%ebx
  402533: 83 e0 07              and    $0x7,%eax
  402536: ff 14 c5 c0 5d 40 00  call   *0x405dc0(,%rax,8)
  40253d: 35 0e 8b 00 00        xor    $0x8b0e,%eax
  402542: 89 c7                 mov    %eax,%edi
  402544: ff 93 c0 5d 40 00     call   *0x405dc0(%rbx)
  40254a: 48 8b 5d f8           mov    -0x8(%rbp),%rbx
  40254e: c9                    leave
  40254f: 05 06 4b 00 00        add    $0x4b06,%eax
  402554: c3                    ret
```

Figure 39: phase6 dispatch 函数汇编

```
(gdb) x/8gx 0x405dc0
0x405dc0 <p6_ops>:      0x00000000004024d0      0x00000000004024b0
0x405dd0 <p6_ops+16>:   0x00000000004024a0      0x0000000000402500
0x405de0 <p6_ops+32>:   0x0000000000402490      0x0000000000402490
0x405df0 <p6_ops+48>:   0x00000000004024f0      0x00000000004024b0
(gdb) info symbol 0x00000000004024d0
p6_op_3 in section .text
(gdb) disassemble 0x00000000004024d0
Dump of assembler code for function p6_op_3:
   0x00000000004024d0 <+0>:     endbr64
   0x00000000004024d4 <+4>:     lea    0x5f1(,%rdi,8),%eax
   0x00000000004024db <+11>:    lea    (%rdi,%rdi,4),%edx
   0x00000000004024de <+14>:    xor    %edx,%eax
   0x00000000004024e0 <+16>:    ret
End of assembler dump.
(gdb) info symbol 0x00000000004024b0
p6_op_2 in section .text
(gdb) disassemble 0x00000000004024b0
Dump of assembler code for function p6_op_2:
   0x00000000004024b0 <+0>:     endbr64
   0x00000000004024b4 <+4>:     mov    %edi,%eax
   0x00000000004024b6 <+6>:     shr    $0x7,%eax
   0x00000000004024b9 <+9>:     xor    %edi,%eax
   0x00000000004024bb <+11>:    add    $0x1d95,%eax
   0x00000000004024c0 <+16>:    ret
```

Figure 40: phase6 gdb 运行结果示例（从 0x405dc0 开始的指针表）

2. 恢复出的计算关系 首先从 challenge 中提取参数和两个函数表下标：

$$x = L_{32}(C \gg 6) \oplus L_{32}(C \gg 32), \qquad i = C \,\&\, 7, \qquad j = (C \gg 3) \,\&\, 7.$$

六个运算函数的计算关系为

$$\begin{aligned}
f_0(x) &= L_{32}((x \oplus \texttt{0x866}) + 2x)\,, \\
f_1(x) &= L_{32}(x + \texttt{0xe554}) \oplus (x \gg 1), \\
f_2(x) &= L_{32}(((x \gg 7) \oplus x) + \texttt{0x1d95})\,, \\
f_3(x) &= L_{32}(8x + \texttt{0x5f1}) \oplus L_{32}(5x), \\
f_4(x) &= \mathrm{ROR}_{32}(x, 1), \\
f_5(x) &= \mathrm{ROL}_{32}(x, 7) \oplus \texttt{0xf72e}.
\end{aligned}$$

其中 ROL_{32} 和 ROR_{32} 分别表示 32 位循环左移和循环右移。

函数指针表的实际顺序为

$$(F_0, \ldots, F_7) = (f_3, f_2, f_1, f_5, f_0, f_0, f_4, f_2).$$

依次调用两个表项后，最终口令为

$$\boxed{a_0 = L_{32}(F_j(F_i(x) \oplus \texttt{0x8b0e}) + \texttt{0x4b06})\,.}$$

3. 控制流图呈现

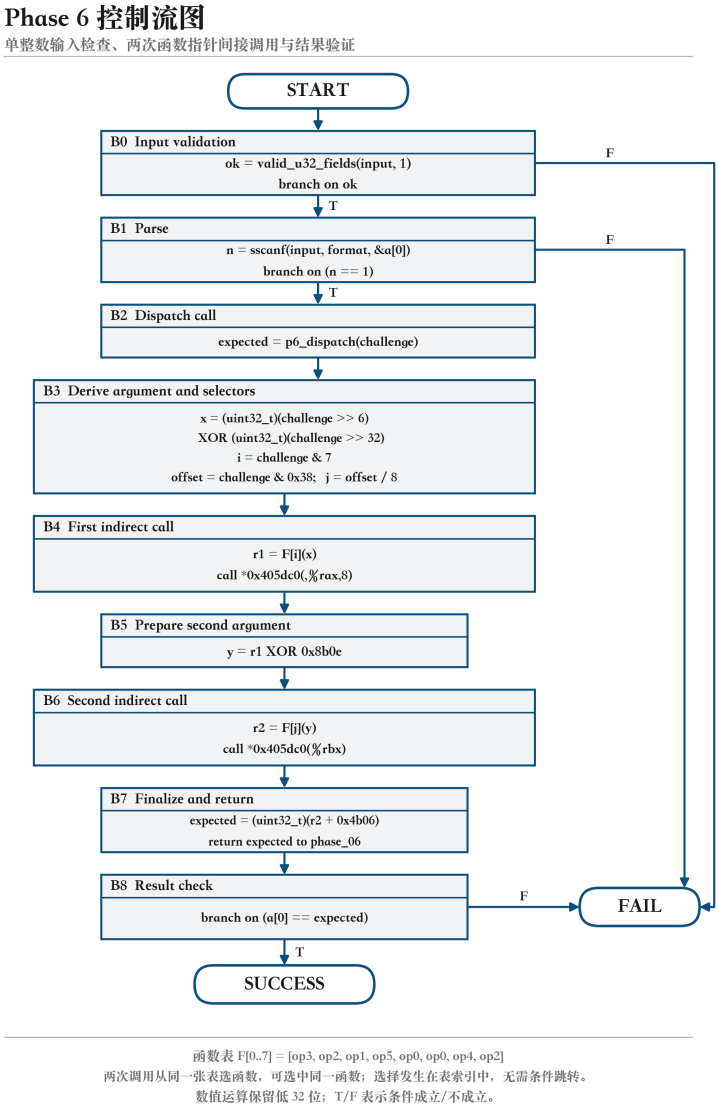


Figure 41: phase6 控制流图

4. 运行成功截图

```
root@ics:~/ics-2026-bomblab# make
python3 public_test.py --phase 6
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -We
li.c -o defuser
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
```

Figure 42: phase6 运行成功截图

6.2 逐行代码推理过程

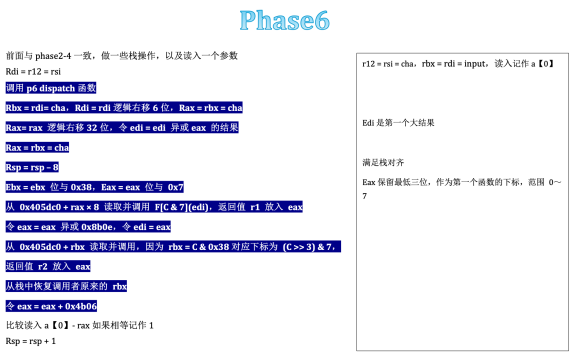


Figure 43: 推理部分

Table 2: Phase 6：函数指针表及关键汇编

下标	函数地址	对应函数	关键汇编与计算关系

0	0x4024d0	p6_op_3	`lea 0x5f1(,%rdi,8),%eax`
			`lea (%rdi,%rdi,4),%edx`
			`xor %edx,%eax`
			返回 $(8x + \texttt{0x5f1}) \oplus (5x)$。

1	0x4024b0	p6_op_2	`shr $0x7,%eax`
			`xor %edi,%eax`
			`add $0x1d95,%eax`
			返回 $((x \gg 7) \oplus x) + \texttt{0x1d95}$。

2	0x4024a0	p6_op_1	`lea 0xe554(%rdi),%eax`
			`shr $1,%edi`
			`xor %edi,%eax`
			返回 $(x + \texttt{0xe554}) \oplus (x \gg 1)$。

3	0x402500	p6_op_5	`rol $0x7,%eax`
			`xor $0xf72e,%eax`
			返回 $\mathrm{ROL}_{32}(x, 7) \oplus \texttt{0xf72e}$。

4	0x402490	p6_op_0	`xor $0x866,%eax`
			`lea (%rax,%rdi,2),%eax`
			返回 $(x \oplus \texttt{0x866}) + 2x$。

5	0x402490	p6_op_0	与下标 4 指向同一函数，运算相同。
6	0x4024f0	p6_op_4	`ror $1,%eax` 返回 $\mathrm{ROR}_{32}(x, 1)$。
7	0x4024b0	p6_op_2	与下标 1 指向同一函数，运算相同。

6.3 额外 seed 验证记录

采用了公开 10 个 seed 以外的 13579 进行测试，测试成功。

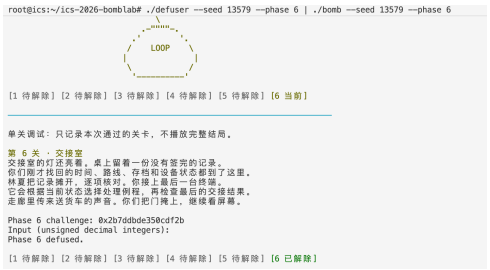


Figure 44: phase6 的额外 seed 验证

7 Honor

7.1 概要分析

1. 关键汇编 Honor 在前六关通过后读取第七行输入，由 `secret_gate` 检查入口格式，调用 `secret_copy` 复制字符串，将运算后的结果再由 `secret_phase` 验证三个整数，根据 challenge、context 计算期望值，调用 `secret_mix` 进行混合计算。关键汇编如下：

```
Dump of assembler code for function secret_gate:
=> 0x00000000004026c0 <+0>:     endbr64
   0x00000000004026c4 <+4>:     push   %rbp
   0x00000000004026c5 <+5>:     xor    %eax,%eax
   0x00000000004026c7 <+7>:     mov    $0x14,%ecx
   0x00000000004026cc <+12>:    mov    %rsp,%rbp
   0x00000000004026cf <+15>:    push   %r14
   0x00000000004026d1 <+17>:    mov    %rdx,%r14
   0x00000000004026d4 <+20>:    push   %r13
   0x00000000004026d6 <+22>:    mov    %rsi,%r13
   0x00000000004026d9 <+25>:    push   %r12
   0x00000000004026db <+27>:    lea    -0xd0(%rbp),%r12
   0x00000000004026e2 <+34>:    push   %rbx
   0x00000000004026e3 <+35>:    mov    %rdi,%rbx
   0x00000000004026e6 <+38>:    mov    %r12,%rdi
   0x00000000004026e9 <+41>:    sub    $0xb0,%rsp
   0x00000000004026f0 <+48>:    rep stos %rax,%es:(%rdi)
   0x00000000004026f3 <+51>:    movl   $0x0,(%rdi)
   0x00000000004026f9 <+57>:    mov    %rbx,%rdi
   0x00000000004026fc <+60>:    call   0x401190 <strlen@plt>
   0x0000000000402701 <+65>:    sub    $0x24,%rax
   0x0000000000402705 <+69>:    cmp    $0x7f,%rax
   0x0000000000402709 <+73>:    jbe    0x402720 <secret_gate+96>
   0x000000000040270b <+75>:    add    $0xb0,%rsp
   0x0000000000402712 <+82>:    xor    %eax,%eax
   0x0000000000402714 <+84>:    pop    %rbx
   0x0000000000402715 <+85>:    pop    %r12
   0x0000000000402717 <+87>:    pop    %r13
   0x0000000000402719 <+89>:    pop    %r14
   0x000000000040271b <+91>:    pop    %rbp
   0x000000000040271c <+92>:    ret
   0x000000000040271d <+93>:    nopl   (%rax)
   0x0000000000402720 <+96>:    mov    %rbx,%rsi
   0x0000000000402723 <+99>:    mov    %r12,%rdi
   0x0000000000402726 <+102>:   call   0x4026a0 <secret_copy>
   0x000000000040272b <+107>:   cmpl   $0x4e45504f,-0xb0(%rbp)
   0x0000000000402735 <+117>:   jne    0x40270b <secret_gate+75>
   0x0000000000402737 <+119>:   mov    %r14,%rdx
   0x000000000040273a <+122>:   mov    %r13,%rsi
   0x000000000040273d <+125>:   lea    -0xac(%rbp),%rdi
```

Figure 45: Honor secret gate 函数汇编示例

```
Dump of assembler code for function secret_phase:
   0x00000000004025e0 <+0>:     endbr64
   0x00000000004025e4 <+4>:     push   %rbp
   0x00000000004025e5 <+5>:     mov    %rsp,%rbp
   0x00000000004025e8 <+8>:     push   %r13
   0x00000000004025ea <+10>:    mov    %rdx,%r13
   0x00000000004025ed <+13>:    push   %r12
   0x00000000004025ef <+15>:    mov    %rsi,%r12
   0x00000000004025f2 <+18>:    mov    $0x3,%esi
   0x00000000004025f7 <+23>:    push   %rbx
   0x00000000004025f8 <+24>:    mov    %rdi,%rbx
   0x00000000004025fb <+27>:    sub    $0x18,%rsp
   0x00000000004025ff <+31>:    call   0x402c10 <valid_u32_fields>
   0x0000000000402604 <+36>:    test   %al,%al
   0x0000000000402606 <+38>:    jne    0x402620 <secret_phase+64>
   0x0000000000402608 <+40>:    xor    %edx,%edx
   0x000000000040260a <+42>:    add    $0x18,%rsp
   0x000000000040260e <+46>:    mov    %edx,%eax
   0x0000000000402610 <+48>:    pop    %rbx
   0x0000000000402611 <+49>:    pop    %r12
   0x0000000000402613 <+51>:    pop    %r13
   0x0000000000402615 <+53>:    pop    %rbp
   0x0000000000402616 <+54>:    ret
   0x0000000000402617 <+55>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402620 <+64>:    xor    %eax,%eax
   0x0000000000402622 <+66>:    lea    -0x24(%rbp),%r8
   0x0000000000402626 <+70>:    lea    -0x28(%rbp),%rcx
   0x000000000040262a <+74>:    mov    $0x40317c,%esi
   0x000000000040262f <+79>:    lea    -0x2c(%rbp),%rdx
   0x0000000000402633 <+83>:    mov    %rbx,%rdi
   0x0000000000402636 <+86>:    call   0x4011e0 <__isoc99_sscanf@plt>
   0x000000000040263b <+91>:    cmp    $0x3,%eax
   0x000000000040263e <+94>:    jne    0x402608 <secret_phase+40>
   0x0000000000402640 <+96>:    mov    %r13d,%ecx
   0x0000000000402643 <+99>:    xor    %edx,%edx
   0x0000000000402645 <+101>:   and    $0x3,%ecx
   0x0000000000402648 <+104>:   cmp    %ecx,-0x2c(%rbp)
   0x000000000040264b <+107>:   jne    0x40260a <secret_phase+42>
   0x000000000040264d <+109>:   mov    %r12,%rsi
```

Figure 46: Honor secret phase 函数汇编示例

2. 恢复出的计算关系 设 C 为 challenge，K 为 context，$L_{32}(x) = x \bmod 2^{32}$，$\oplus$ 表示按位异或。首先计算

$$u = L_{32}(C \gg 32) \oplus L_{32}(C), \qquad v = L_{32}(K \gg 32) \oplus L_{32}(K).$$

三个整数依次为

$$\begin{aligned} a &= K \,\&\, 3, \\ x &= u \oplus v, \\ m &= L_{32}(((x \gg 7) \oplus \texttt{0x4500}) + (x \oplus \texttt{0x720})), \\ b &= m \oplus ((a \cdot \texttt{0x720}) \oplus L_{32}(K \gg 16)), \\ c &= b \oplus L_{32}(v + a + \texttt{0x4500} + (u \gg 7)). \end{aligned}$$

最终将 a, b, c 按无符号十进制格式写在 `OPEN` 后，整数之间以空格分隔。

3. 控制流图呈现

因篇幅问题逐行代码解释过于冗长，此处 honor 的解析由控制流图替代。

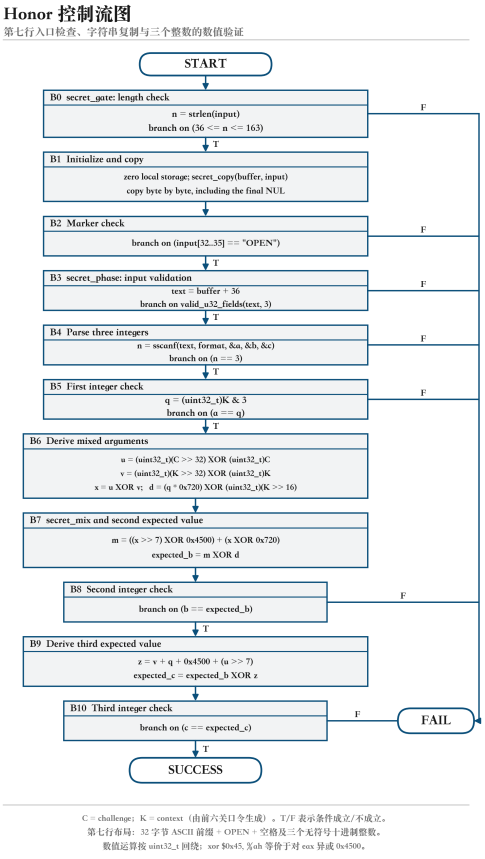


Figure 47: Honor 控制流图

4. 运行成功与额外 seed 截图

采用了公开 10 个 seed 以外的 13579 进行测试，测试成功。

```
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10 (optional)
```

Figure 48: Honor 运行成功截图

```
root@ics:~/ics-2026-bomblab# make
{ ./defuser --seed 13579; ./defuser --secret --seed 13579; } | ./bomb --seed 13579
python3 public_test.py --secret
'日志看到了，新存档正常。谢谢你们，剩下的维护我来接。'

实验室的灯还亮着，外套仍搭在椅背上。你确认文件都保存了，合上电脑。
林夏揉了揉眼睛：'还活着，精神状态另说。'
你把椅子推回桌下：'事已至此，先睡觉吧。'
你们关掉灯，走出了实验室。

--------------------------------------------------------------------------------

谢谢你远道而来，与我相遇。

晚安。

— sky

--------------------------------------------------------------------------------
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10 (optional)
```

Figure 49: Honor 额外 seed 验证截图

8 思考与总结

完整代码 test 后的结果

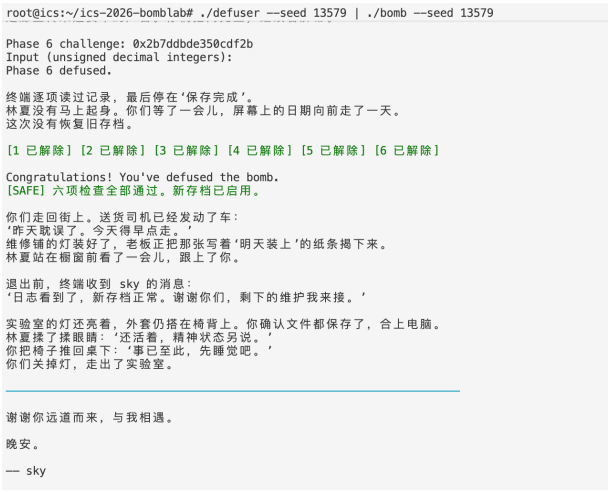


Figure 50: 6 关通关

每一个 phase 的收获 1. 把握中间量很重要，适时的找到反复出现的寄存器，把它对应到 c 语言中可以省去很多繁琐的步骤

2. 数组元素和元素下标不一样，要看后续具体指令：如果指的是地址，对应的是具体元素的数值；如果涉及循环，+1，是否有上限，那代表的是 a[i] 的那个 i 下标。有的时候 lea 只是数据运算，有的时候是地址运算，要看后续指令以及运算是否存在什么特殊的计算关系（比如字节数量，比如后面有 sscanf 函数）。

3. 注意函数的承接关系，已经变更过的要承接下去；注意 case 入口由谁表示，要有判定。

4. 区分中点与区间折半，区间折半只是逻辑右移，但是要加上下界才是下一个新的中点；从下至上推的时候要注意递归之前保存本层信息的栈操作，在每一层更新区间的时候都已经记录了当前的 mid 和 key，这样在从下至上计算的时候能够直接调用，而在缩小区间和改变中点从上到下运作的过程中，是不经历计算的！因为直接跳转到函数开头了。

5. 指针表是一连串有地址，有数据的节点连成的，要确定起点，方向，以及走的步数，每一步会有对应结果 rax 的运算，也会取出对应地址的 field8 位置放入到提前准备好的 a 中。这样最后得到累计运算的 rax 以及不断被覆盖的 a，和 input 进行对比。

6. 指针表出现的典型形式：`call *某个新地址（，rax，8）`因为在指针表中每个元素对应占 8 个字节，所以这就代表了要对应到里面的某一个元素。每个元素有地址，有对应的函数，需要 gdb 进行动态拆解，因而对应的 rax 也就是下标。字节偏移对应的下标和内部实际顺序 caseid 不一定一一对应，所以要注意地址背后实际的函数如何操作。

诚信说明： 详细阅读了深入理解计算机系统第三版对应部分的解析，并完成了相应习题来加深对代码的理解；使用 AI 大模型来 debug 运行过后报错的部分，并借此把自己做的流程图润色为更严谨、更美观的控制流图，使报告呈现更加美观有条理。